DL Complexity: One Interface, Different Measures of Deep Learning Model Complexity

Alyabushev, Dementiev, Kurdyukov, Chirkov
[affiliations]

Draft of 21 October 2026

Abstract

Classical model-selection criteria such as AIC, BIC and R^2 are hard to apply to large deep learning models, and the parameter count of an overparameterized network says little about how well it generalizes. Complexity measures that do apply to deep networks live in separate papers with different inputs, scales and assumptions, so they are rarely run side by side. We present DL-Complexity, a PyTorch library that evaluates model complexity through one calling convention while preserving the meaning of each measure. A model with its likelihood and data goes through a common `check` / `estimate` protocol and returns a structured `ComplexityResult` whose named quantities carry an explicit unit and normalization, together with the assumptions, configuration and provenance of the run. Methods are grouped by what they measure: information criteria (AIC, BIC, HQIC, WAIC), code lengths (two-part and prequential MDL, with a bridge from Bayesian estimators such as variational coding and KFAC-Laplace evidence), and risk bounds (compression-based generalization bounds). One calling convention does not imply one scale or a universal ranking, so the library never compares raw values across measures; we compare measures by how well each tracks test negative log-likelihood and the generalization gap on MLP and CNN families. **[headline benchmark result, e.g. which measure attains the highest rank correlation with the generalization gap and how it compares with the parameter-count baseline.] [one sentence on cost: which measures remain practical as model size grows.]** The library is released as the open-source Python package `dl_complexity`.

1 Introduction

Which of two trained models is more complex? For classical statistical models the answer is routine: AIC, BIC, R^2 and its variations are the standard tools for model selection. For large deep learning models they mostly cannot be applied as they stand. The obvious replacement, the parameter count, fares poorly: modern networks routinely have more parameters than training examples and still generalize, so the number of parameters says little about how complex a trained model really is [7, 6].

Description length offers a principled alternative. In the minimum description length (MDL) view, a model is good if it compresses the data, and its complexity is the length of the code for the model plus the code for the data given the model [1]. The Bayesian view reaches a related quantity from the other side: complexity is the penalty that arises when the model evidence is approximated, and the two views are tied together through the code length of the Bayesian mixture [1, Sect. 2.6.3]. Both views have been turned into concrete estimators for neural networks: variational coding [2], Laplace approximations with Kronecker-factored curvature [3], prequential codes learned sequentially with replay [4], and compression-based generalization bounds [5]. Classical criteria such as AIC, BIC, HQIC and WAIC sit at the simple end of the same family.

However, these measures have been developed in isolation. They need different inputs: AIC and BIC need a total log-likelihood, a parameter count and a sample count, WAIC needs pointwise log-likelihoods over posterior samples, and prequential codes need a training procedure. They also live on different scales: information criteria on their own native scale, code lengths in nats or bits, and risk bounds on a risk scale that holds only under assumptions. A shared interface can therefore mislead if it hides these differences, and a loss is not automatically a negative log-likelihood. There is also no shared benchmark, so it is unclear which measure to use for which model. Criteria of the AIC and BIC family in particular rest on a regular finite-dimensional maximum-likelihood setting and are not a universal "true complexity" of a network; any comparison has to state that domain.

In this work, we introduce DL-Complexity, a PyTorch library built around four design principles. First, **one protocol**: every measure exposes `check`, which inspects whether the inputs are ready, and `estimate`, which computes the measure. Second, **each measure keeps its own meaning**: results are structured objects with named quantities, each with a unit and a normalization, and the library performs no cross-measure ranking. Third, **explicit statistical assumptions**: the likelihood is a separate object, since a loss is not automatically a negative log-likelihood. Fourth, **explicit fitting**: the library never trains a model implicitly, and `check` establishes the readiness of the inputs, not the optimality of the parameters or the validity of asymptotic assumptions.

On top of the library, we ask: how well does each measure track test NLL, the generalization gap and the stability of the model when the sample size, architecture and regularization change? We answer by comparing how each measure orders trained models, not by comparing raw values across measures (Figure 1).

Our contributions are as follows:

1. **Scale-preserving interface.** We define a PyTorch-facing interface (`ModelContext`, `DataSource`, `check` / `estimate`, `ComplexityResult` with `Quantity(value, unit, normalization)`) and a `ComplexitySuite` that runs named estimators without implicit fitting, shared cache or cross-measure ranking.

2. **Extensible estimator hierarchy.** We provide a typed hierarchy rooted in `ComplexityEstimator[TInputs]` with branches for information criteria, Bayesian estimators, code lengths, generalization bounds and diagnostics, so that a new measure needs only an input type and an `_estimate` method.

3. **Estimators with analytic validation.** We implement the information criteria on shared `FitStatistics`, validate BIC against an analytic reference on known-variance Gaussian cases, and extend the library to code-length and bound estimators. **[status of the remaining estimators at submission.]**

4. **Benchmark and empirical comparison.** We fix seeds, splits and training budgets on synthetic data, MNIST and Fashion-MNIST, and compare how each measure tracks test NLL and the generalization gap, together with time and memory cost. **[grid size and main finding.]**

5. **Open-source package.** We release `dl_complexity` with unit and integration tests and documentation.

The rest of the paper is organized as follows. Section 2 describes the library and each family of measures. Section 3 specifies the benchmark, Section 4 reports the results, Section 5 discusses the limits of the measures, and Section 6 concludes.

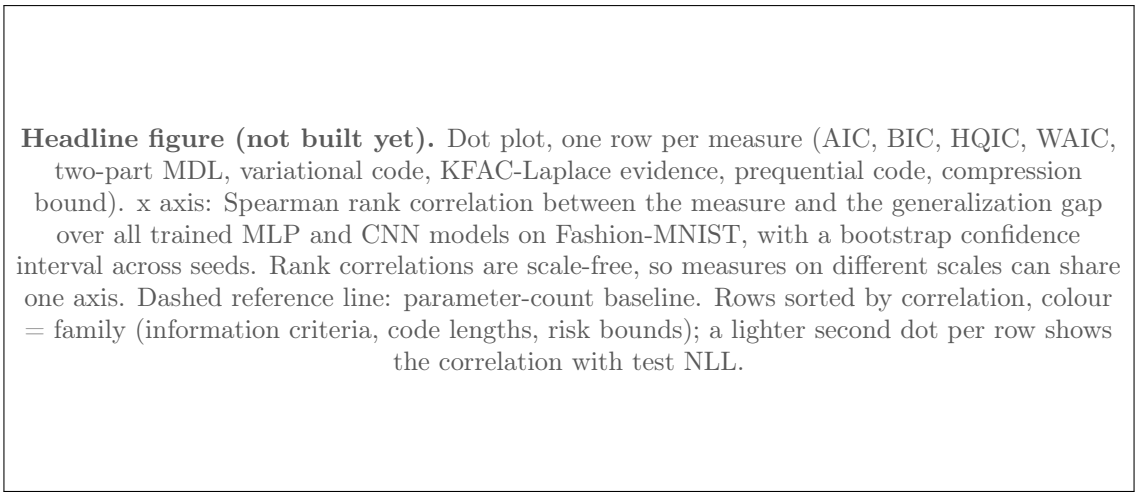


Figure 1: Which complexity measure tracks generalization? **[one-sentence message once the data exist.]**

2 Methods

2.1 Library design

(may be it is better to crop this part from the paper)

`dl_complexity` evaluates the complexity of an ordinary PyTorch `nn.Module` in three steps (Figure 2): the model and its data are wrapped, an estimator checks and estimates, and the result comes back as a structured object. Prepared inputs can skip the model.

Inputs. A `ModelContext` holds the model, its likelihood (a log-density) and, separately, the loss, so that the relation between the loss and the negative log-likelihood is always stated explicitly. It can also hold a `TrainingRecipe`, which stores instructions for training but never triggers it. A `DataSource` provides batches, the sample count and a data identity. Method-specific typed inputs, such as `FitStatistics` (total log-likelihood, parameter count, sample count) and `PosteriorLogLikelihood`, can be passed instead of a model; posterior samples are always supplied explicitly.

Protocol. Every estimator exposes `check(...)`, which returns a `RequirementReport`, and `estimate(...)`, which returns a `ComplexityResult`. Both accept either a context together with data (the direct route) or prepared `inputs` (the prepared route). The same prepared statistics can be reused for AIC, BIC and HQIC without another pass through the model.

Result. A `ComplexityResult` records the method and its kind, a set of named quantities `Quantity(value, unit, normalization)` with a `primary_name`, the assumptions and configuration of the run, its provenance and the elapsed time.

Estimator hierarchy. All estimators derive from `ComplexityEstimator[TInputs]`, with the branches `InformationCriterion`, `BayesianEstimator`, `CodeLengthEstimator`, `GeneralizationBoundEstimator` and `DiagnosticEstimator`. Implementing a new method requires declaring `input_type` and implementing `_estimate(inputs)`; optional `_check_inputs` adds readiness checks, and a direct route adds `_check_context` and `_prepare_context`. Model transformations and model encodings are separate interfaces, `ModelTransform.apply(model)` and `ModelCodec.encode(model)` / `decode(artifact)`, not estimator subclasses.

Suites. A `ComplexitySuite` runs named estimators and returns an `EvaluationOutcome` per method. It performs no implicit fitting, keeps no shared cache and does no cross-measure ranking.

Scales. The measures fall into three families that are reported on different scales (Table 1). Sharing a calling convention does not make them comparable value by value.

Table 1: Measure families and their scales. One calling convention does not imply one scale or a universal ranking.

Family	Measures	Scale	Reads as
Information criteria	AIC, BIC, HQIC, WAIC	`native_ic`, normalization `total`	Fit plus a complexity penalty
Code lengths	Two-part and prequential MDL; Bayesian estimators through a bridge	nats or bits	Description length of the model and the data
Risk bounds	Compression-based generalization bounds	Risk scale, under assumptions	Generalization guarantee

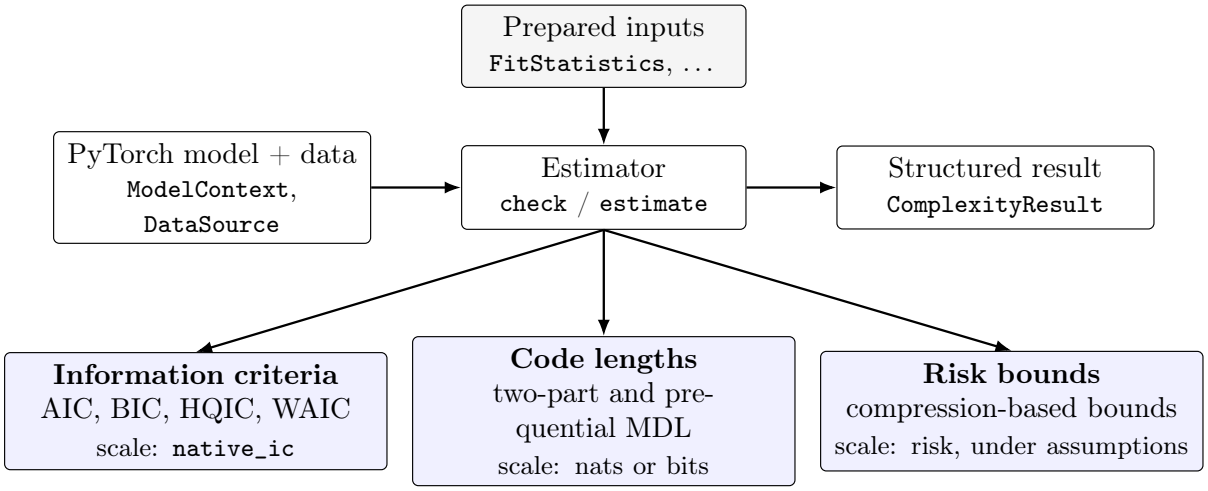


Figure 2: Architecture of `dl_complexity`. A PyTorch model with its data (or prepared inputs) goes through the `check` / `estimate` protocol of an estimator and returns a `ComplexityResult`. The estimators are grouped into three families by what they measure and the scale they report.

2.2 Information criteria

AIC, BIC and HQIC share one preparation step: an internal `FittedCriterion` obtains `FitStatistics`, that is, the total log-likelihood $\ell = \log L(\hat{\theta})$ at the fitted parameters, the parameter count k and the sample count n, either from a model and data or from prepared inputs. BIC is computed as

$$\mathrm{BIC} = \underbrace{-2 \log L(\hat{\theta})}_{\text{deviance}} + \underbrace{k \log n}_{\text{penalty}}. \tag{1}$$

The library returns `bic`, `deviance` and `penalty` as quantities, with unit `native_ic` and normalization `total`. After preparation the formula costs $O(1)$; the direct route additionally evaluates the model. BIC assumes a regular finite-dimensional maximum-likelihood setting, and `check().ready` does not establish MLE optimality or the asymptotic assumptions of BIC. WAIC additionally needs posterior samples with pointwise log-likelihoods, passed as `PosteriorLogLikelihood`. **[formulas and sign conventions for AIC, HQIC and WAIC; status of WBIC; effective parameter count for neural networks.]**

2.3 Code lengths

Code-length estimators report a description length in nats or bits.

Two-part MDL. The code length is the sum of the code for the weights and the code for the data given the weights, $C_{2\mathrm{P}} = L(\theta) + L(\mathcal{D} \mid \theta)$ [1]. **[coding scheme for the weights and the resulting unit.]**

Table 2: Planned summary: one row per measure. Cells marked TBD are filled as the estimators are completed.

Measure	Required inputs	Main assumptions	Cost
BIC (AIC and HQIC share `Fit Statistics`)	Total log-likelihood, parameter count, sample count	Regular finite-dimensional MLE setting; `check` does not verify them	$O(1)$ after preparation; the direct route also evaluates the model
WAIC	`PosteriorLogLikelihood`: posterior samples supplied explicitly	[]	[]
Two-part MDL	[]	[]	[]
Variational code, KFAC-Laplace	[]	[]	[]
Prequential code	Training procedure and data blocks []	[]	[]
Compression bound	[]	[]	[]

Bayesian estimators and the bridge to MDL. Variational coding [2] and KFAC-Laplace evidence [3] are Bayesian estimators, but the interface needs them as code lengths. The planned bridge rests on two facts: the negative log marginal likelihood is the code length of the Bayesian mixture [1, Sect. 2.6.3], and the variational objective is an upper bound on it that can be read as a description length of the weights and the data [2]. **[exact conversion for each Bayesian estimator and its unit; check against Sect. 2.6.3 of the MDL tutorial and the variational-coding paper.]**

Prequential coding. The data are encoded as a stream: with the training set split into blocks, the prequential code length is the total cost of predicting each block from a model trained on the previous ones,

$$C_{\text{preq}} = \sum_{t=1}^{T} -\ln p_{\theta_{<t}}(\mathcal{D}_t \mid x_{\mathcal{D}_t}) \,, \tag{2}$$

where $\theta_{<t}$ is the model fitted to blocks $1, \ldots, t-1$. The variants of [4] (blockwise, online, replay, calibration) are to be implemented and compared. **[block schedule, replay policy, calibration and treatment of the first block; the exact set of variants is fixed against the paper.]**

2.4 Risk bounds

Compression-based generalization bounds [5] turn the compressibility of a trained network into a bound on its risk. They live in `GeneralizationBoundEstimator`, with network compression expressed through the separate `ModelTransform` and `ModelCodec` interfaces. The result is on a risk scale and is valid only under the stated assumptions; it is neither a code length nor an information criterion. **[compression schemes, how a compressed network is turned into a bound, and the assumptions that must hold.]**

3 Experimental setup

Models. Families of MLPs and CNNs of different width, depth and regularization.
Data. Synthetic data first, then MNIST and Fashion-MNIST.
Protocol. Every model is trained by the benchmark runner with a fixed seed, split and training budget; the library itself never fits a model implicitly. Each run stores its configuration, time and memory.

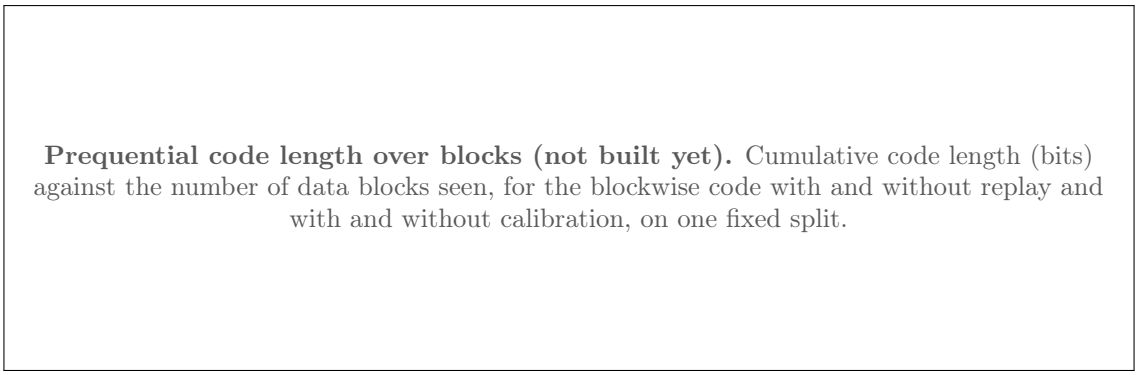


Figure 3: Prequential code length over data blocks. **[message.]**

Metrics. Test NLL, generalization gap, time, memory, and the Spearman rank correlation of each measure with generalization. Measures are never compared by raw value across scales. **[grid of widths, depths, regularizers and sample sizes; number of seeds; training budget; definition of the generalization gap.]**

4 Results

4.1 BIC against an analytic reference

As a first end-to-end validation, we take a Gaussian model with known variance and four observations,

$$y = (1, 2, 4, 5), \quad y_i \sim \mathcal{N}(\mu, 1), \quad \hat{\mu} = \bar{y} = 3, \quad n = 4, \quad k = 1, \tag{3}$$

for which $\sum_i (y_i - \hat{\mu})^2 = 10$ and $\ell(\hat{\mu}) = -\frac{4}{2}\log(2\pi) - \frac{10}{2}$. The model is `Linear(1, 1, bias=False)` on unit inputs with the weight set analytically to 3, so no hidden training takes place. BIC is computed through both public routes, from the model and data and from `Fit Statistics`, and compared with the analytic formula. Both routes agree with the independent reference to 10^{-12}, with unit `native_ic` and normalization `total`. Five known-variance Gaussian cases pass the analytical tests of the information criteria.

This is an integration check, not a full benchmark and not a proof for arbitrary neural networks. Here the MLE is known from the analytic solution; in general `check().ready` does not establish it.

4.2 Complexity measures versus generalization

Per-measure scatter plots (Figure 4) and the rank correlations summarized in Figure 1. **[result.]**

4.3 Training-set size

The behaviour of each measure as the training set grows (Figure 5) tests the stability part of the research question. **[result.]**

4.4 Cost

Time and memory of each measure against its rank correlation with generalization (Figure 6). **[result.]**

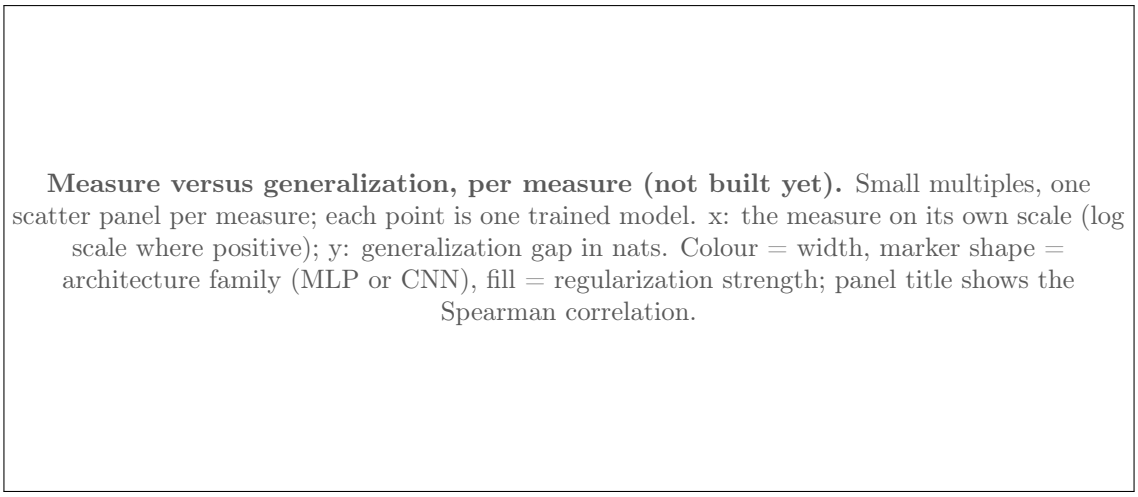


Figure 4: Complexity measure versus generalization gap on Fashion-MNIST. Synthetic data and MNIST in the appendix. **[message.]**

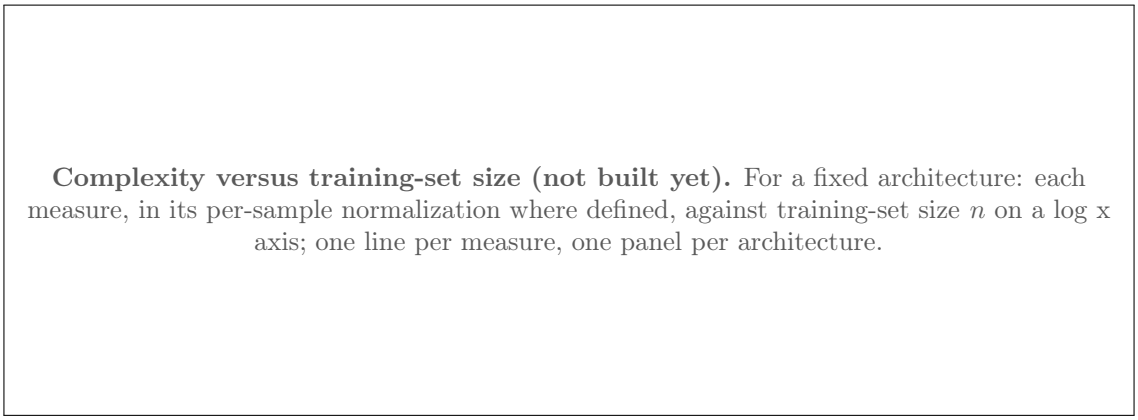


Figure 5: Normalized complexity against training-set size. **[message.]**

5 Discussion and limitations

Where each measure fails; which assumptions break for neural networks; why one calling convention does not give one scale or a universal ranking, and why AIC- and BIC-like criteria are not a universal complexity; limits of small MLPs and CNNs on MNIST-scale data. **[write after results.]**

6 Conclusion

What was built (one scale-preserving interface, three families of measures, one benchmark, the package `dl_complexity`), what was found, and what comes next. **[write after results.]**

References

[1] P. Grünwald. A tutorial introduction to the minimum description length principle. 2004. https://arxiv.org/abs/math/0406077

[2] A. Graves. Practical variational inference for neural networks. In *NeurIPS*, 2011. https://papers.nips.cc/paper/4329-practical-variational-inference-for-neural-networks

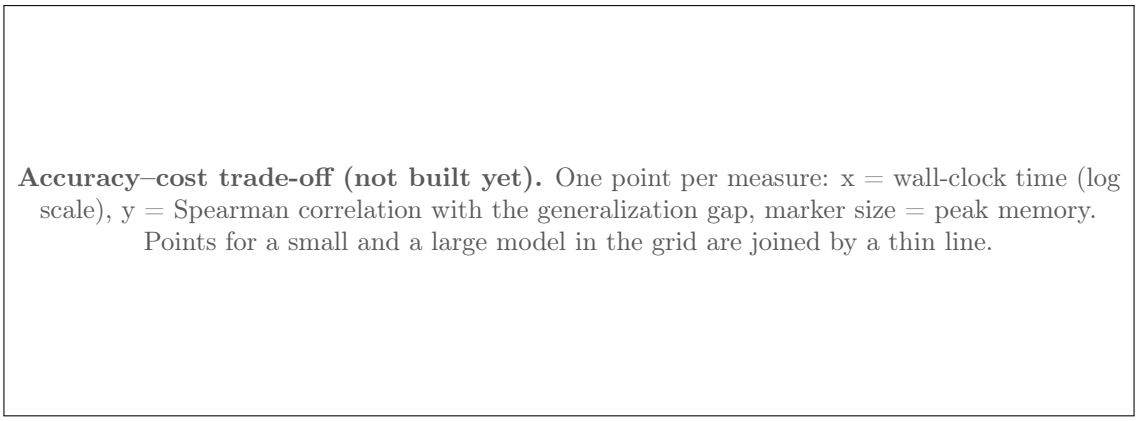


Figure 6: Accuracy–cost trade-off of the measures. **[message.]**

[3] H. Ritter, A. Botev, D. Barber. A scalable Laplace approximation for neural networks. In *ICLR*, 2018. https://openreview.net/forum?id=Skdvd2xAZ

[4] J. Bornschein, Y. Li, M. Hutter. Sequential learning of neural networks for prequential MDL. 2022. https://arxiv.org/abs/2210.07931

[5] S. Arora et al. Stronger generalization bounds for deep nets via a compression approach. In *ICML*, 2018. https://proceedings.mlr.press/v80/arora18b.html

[6] X. Hu, L. Chu, J. Pei, W. Liu, J. Bian. Model complexity of deep learning: a survey. 2021. https://arxiv.org/abs/2103.05127

[7] C. Zhang et al. Understanding deep learning requires rethinking generalization. **[verify venue and year; not in the repository literature index.]**

[add citations for AIC, BIC, HQIC and WAIC (cited in the text, not in the repository index).]